



Module 8: Support vector machines and kernels

5XSL0: FUNDAMENTALS OF MACHINE LEARNING

Dr. Ir. Luis A. Zavala Mondragón

Prof. Fons van der Sommen

Prof. Uzay Kaymak

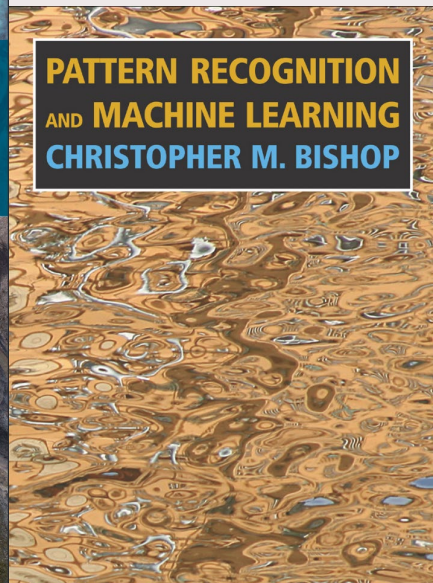
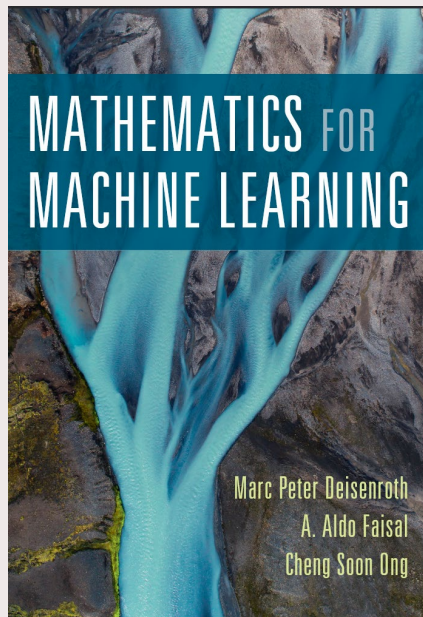
Cover image generated
with MS365 copilot ver.
19.2509.54051.0

Dept EE – Signal Processing Systems group

Version Sept 2026

Credits

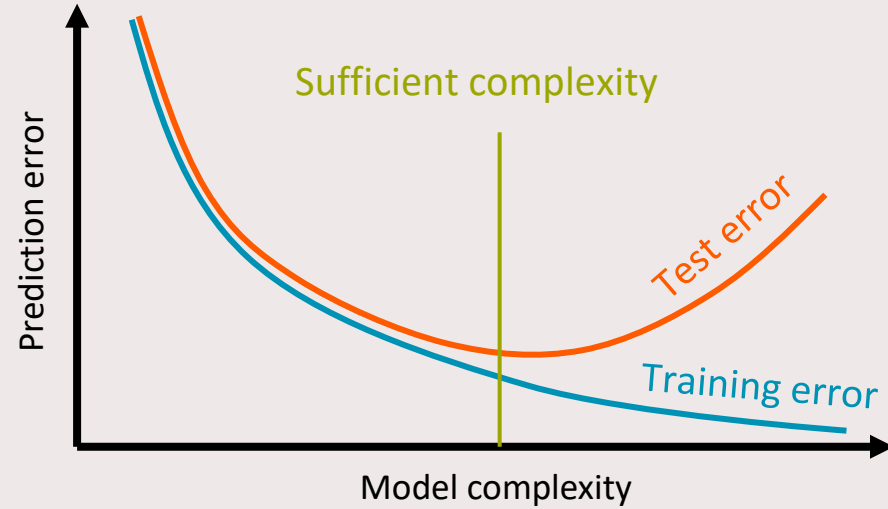
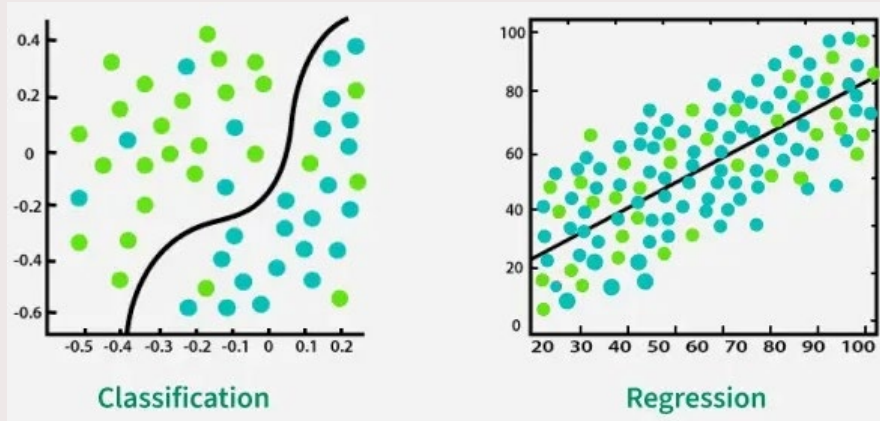
- Slides based on the lecture 5XSLO by
Prof. Fons van der Sommen
- Also on the books
 - Mathematics for machine learning.
 - Pattern recognition and machine learning.



Learning objectives

- Explain what is a maximum margin classifier and how it differs from a regular linear classifier
- To understand the concept of the dual formulation of an SVM and how it allows for integrating kernels.

Last time



Mathematical notation

Symbol	Description
$x, \mathbf{x}, \mathbf{X}$	Scalar variable, vector and matrix
(x_i, t_i)	i -th data point
t_i	i -th class label, for SVM $t_i \in [-1, 1]$
$\mathbf{w}$	Model weights
$\phi_i(x), \phi_i, \Phi$	Mapping functions, array of bases
$\mathcal{L}(\cdot)$	Loss function

Introduction

Warmup

- What is the meaning of the inner product between two vectors?
- What is the equation for a 2 feature linear classifier?
- For such function. How would you find the slope and the intercept values?

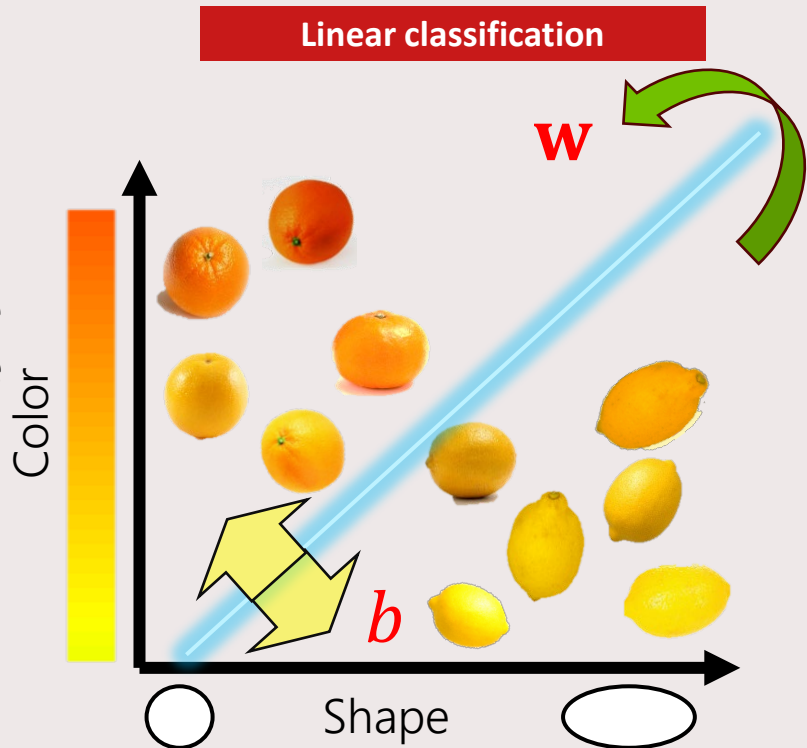
Introduction to linear classification models

The generalized linear model is given by

$$y(\mathbf{x}) = f(\mathbf{w}^T \mathbf{x} + b)$$

Where $f(\cdot)$ is the *activation* function, $\mathbf{w}$ are the *model weights* and w_0 is known as the *bias/threshold*.

- $\mathbf{w}$ sets orientation of the decision surface
- b sets location of the decision surface

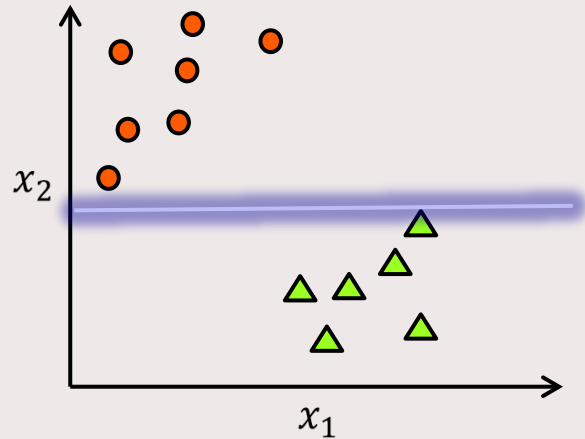


What are we studying today?

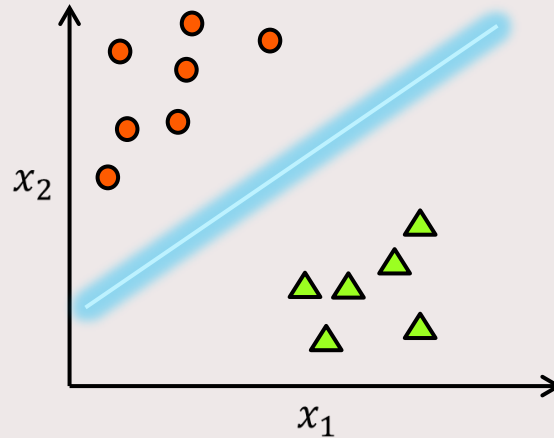
- **Maximum margin classifiers**
- **Kernels and mapping functions**

Which classifier generalizes better?

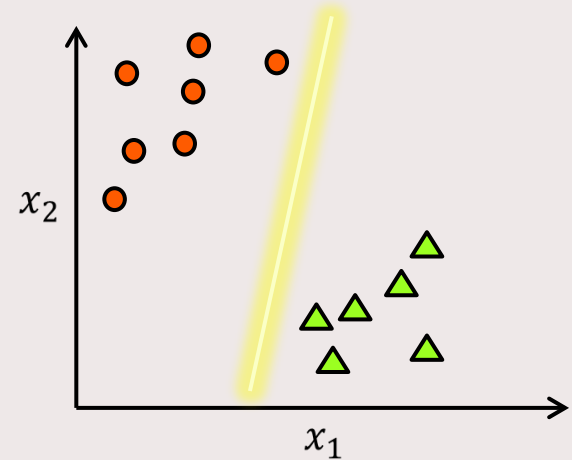
Classifier A



Classifier B

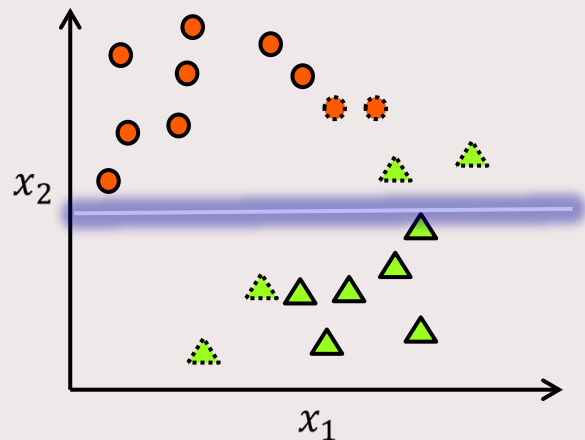


Classifier C

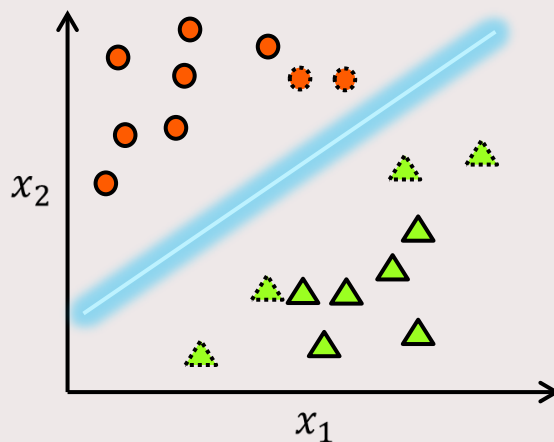


Which classifier generalizes better?

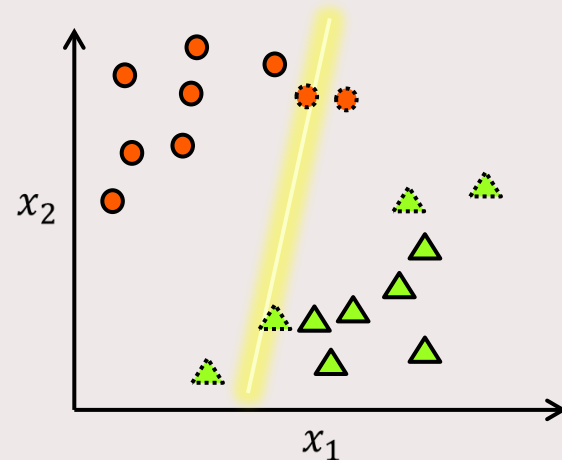
Classifier A



Classifier B



Classifier C



Sample pos. class



Sample neg. class

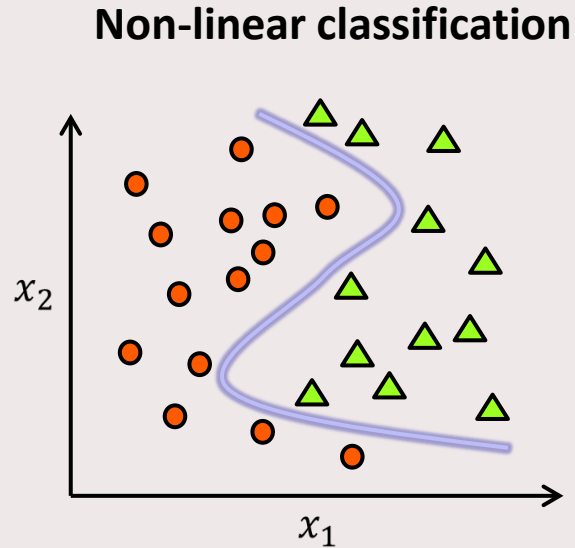
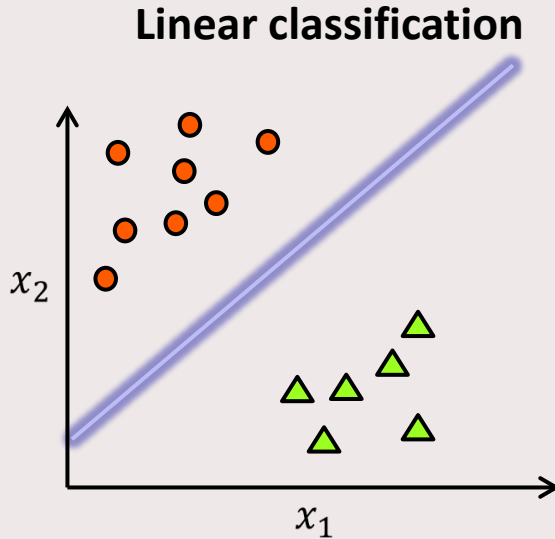


New sample pos. class



New sample neg. class

Linear vs non-linear classification problems



Sample – pos. class



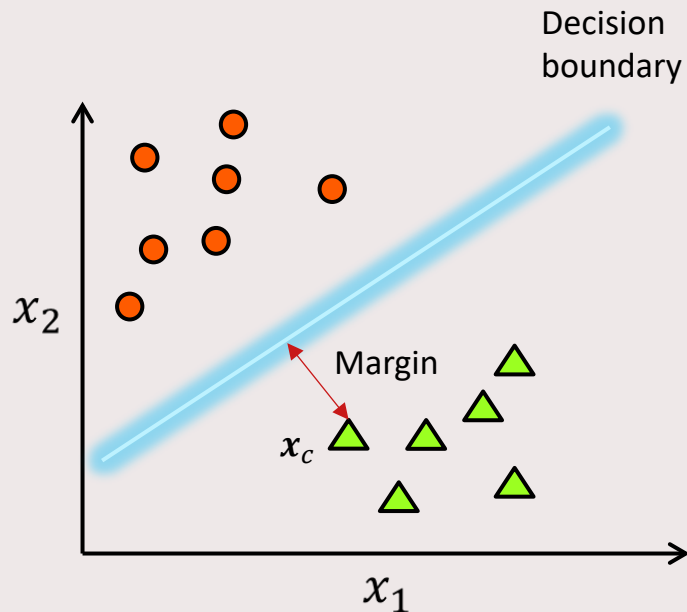
Sample – neg. class

Maximum margin classifiers

- **Hard maximum margin classifier:** There is a hyperplane that can separate all the training samples.
- **Soft maximum margin classifier:** Maximizes distance between datapoints and decision boundaries, but crossings between data points and hyperplane are allowed. This is most commonly known as **support vector machine (SVM)**.
- **Kernel trick.** Allow linear classifiers to define non-linear classification boundaries by mapping data to a different space efficiently.

Hard MMC

Optimal margin classifier



- **Geometric margin:** Distance from decision boundary to closest datapoints.
- **Support vectors:** Training observations whose distance to the decision boundary is equal to the margin.

Key idea of maximum margin classifier (MMC)

- Maximize the margin between the closest points and the decision boundary.
- The support vectors are **the data points that are the *hardest* to classify**.
- This should increase the robustness of the model!
- Core assumption: **ALL** the points of the two classes always can be separated with a hyperplane.

Optimal margin classifier

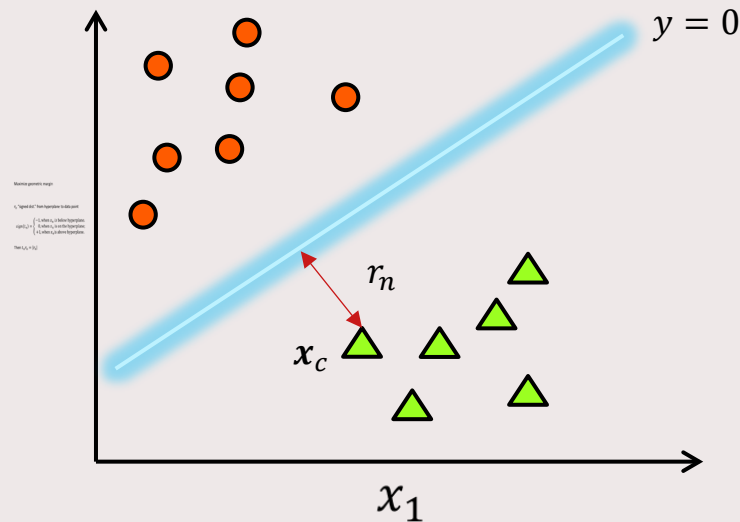
Maximize geometric margin

$$\arg \max_{w,b} \left\{ \min_n [t_n r_n] \right\}$$

r_n “signed dist.” from hyperplane to data point

$$\text{sign}(t_n) = \begin{cases} -1, & \text{when } x_n \text{ is below hyperplane.} \\ 0, & \text{when } x_n \text{ is on the hyperplane;} \\ +1, & \text{when } x_n \text{ is above hyperplane.} \end{cases}$$

Then $t_n r_n = |r_n|$



Distance from a point to the decision surface

Let a linear activation function $f(\mathbf{x}) = \mathbf{x}^T \mathbf{w} + b$
 $y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b.$

Here, $\mathbf{x}$ can be expressed by

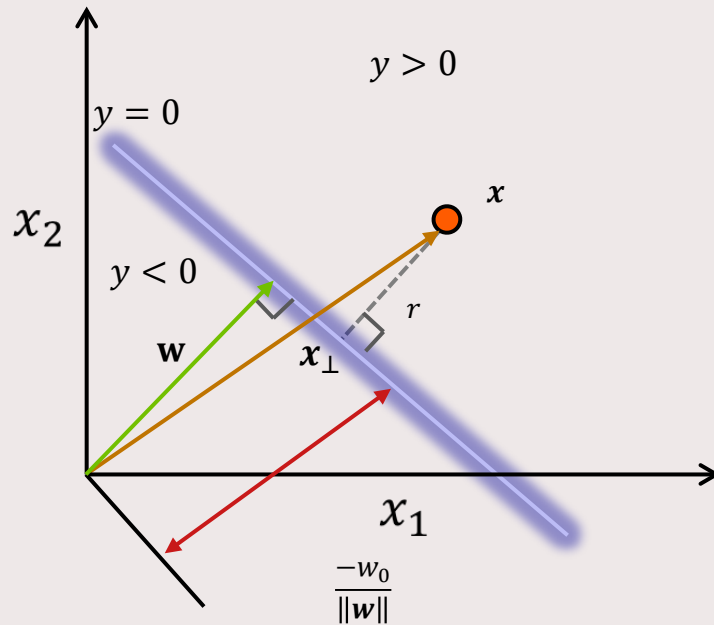
$$\mathbf{x} = \mathbf{x}_\perp + r(\mathbf{w}/\|\mathbf{w}\|)$$

Mult. both sides by $\mathbf{w}^T$ and adding b

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = (\mathbf{w}^T \mathbf{x}_\perp + b) + r \|\mathbf{w}\|$$

Here, $(\mathbf{w}^T \mathbf{x}_\perp + b) = 0$. Consequently

$$r = \frac{y(\mathbf{x})}{\|\mathbf{w}\|}$$



Finding weights of optimal margin classifier

Based on the previous slide, we rewrite the problem

$$\arg \max_{\mathbf{w}, b} \left\{ \min_n [t_n r_n] \right\}$$

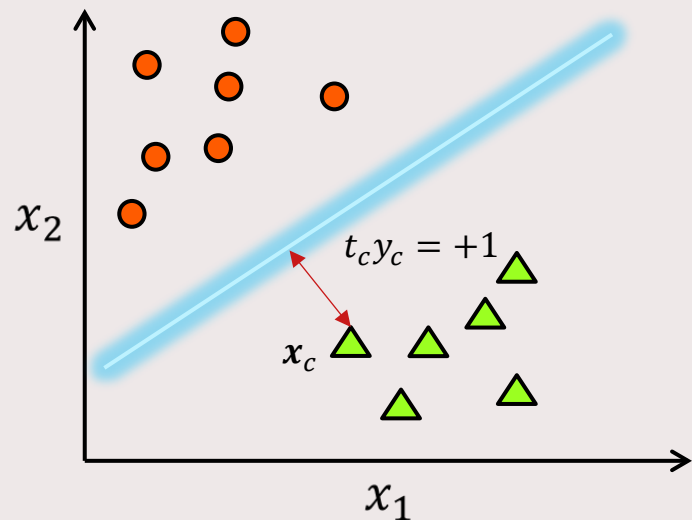
To simplify the problem, we assign $t_c y(\mathbf{x}_c) = 1$ the closest point to the decision boundary (i.e. $t_c r(\mathbf{x}_c) = 1/\|\mathbf{w}\|$)

This leads to the simplified problem

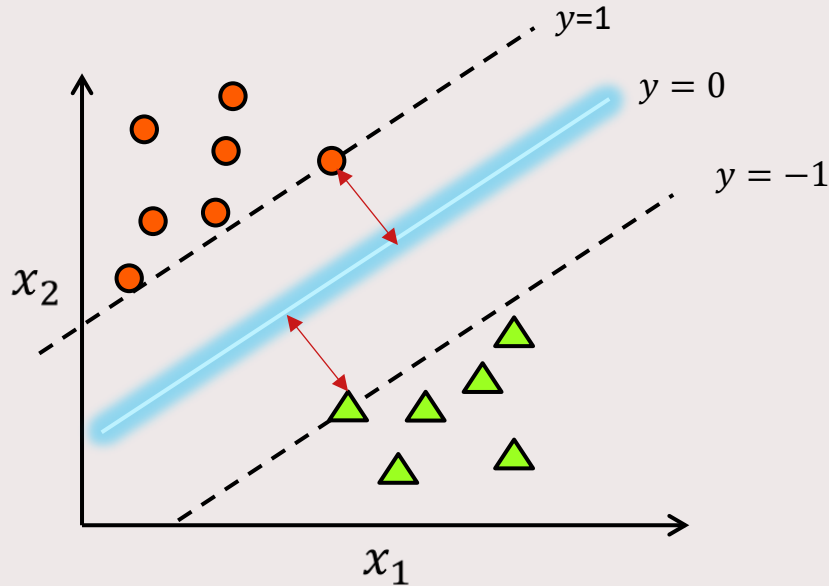
$$\arg \max_{\mathbf{w}, b} \left\{ \frac{1}{\|\mathbf{w}\|} \right\}$$

Subject to

$$t_n (\mathbf{w}^T \mathbf{x}_n + b) \geq 1$$



Visualization of the support vectors



Only the “hardest” data points move the decision boundary

They are called **support vectors**, hence the name support vector machine

Finding weights of optimal margin classifier (2)

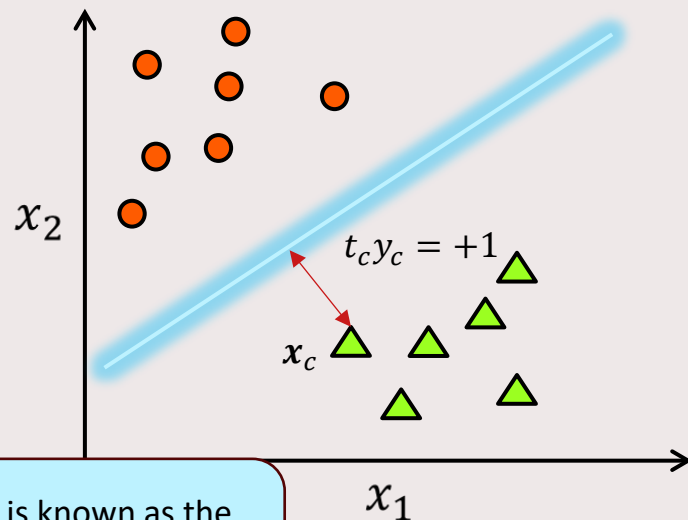
Maximizing $\frac{1}{\|\mathbf{w}\|}$ is equivalent to minimize $\|\mathbf{w}\|^2$. Then the optimization problem becomes

$$\arg \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

Subject to

$$t_n(\mathbf{w}^T \mathbf{x}_n + b) \geq 1$$

This problem is convex and can be solved with quadratic programming



This is known as the “primal” formulation of the SVM

Introduction to dual representation

The formulation beforehand allows to find the weights of the MMC.

It works, but it can be reformulated in such a way that the alternative formulation allow more efficiently using **kernels**.

The new representation is referred to as the **dual representation** of the MMC presented earlier.

Dual representation (1/3)

The Lagrange multiplier of the MMC is defined by

$$\mathcal{L}(\mathbf{w}, b, \mathbf{a}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{n=0}^{N-1} a_n \{t_n(\mathbf{w}^T \mathbf{x}_n + b) - 1\}$$

Where a_n are the Lagrange mults. Solving $\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0$ for $\mathbf{w}$ leads to

$$\mathbf{w} = \sum_{n=0}^{N-1} a_n t_n \mathbf{x}_n$$

This means that **the weights are linear combinations of the data points**. This is known as **the representer theorem**.

Note that not all the constraints $t_n(\mathbf{w}^T \mathbf{x}_n + b) \geq 1$ are *active*. Therefore most entries of a_n are zero

Dual representation (2/3)

Solving $\frac{\partial \mathcal{L}}{\partial b} = 0$ for b leads to

$$0 = \sum_{n=0}^{N-1} a_n t_n$$

Dual representation (3/3)

Substituting these findings in $\mathcal{L}(\mathbf{w}, b, \mathbf{a})$ leads to the **dual representation**

$$\mathcal{L}(\mathbf{a}) = \sum_{n=0}^{N-1} a_n - \frac{1}{2} \sum_{n=0}^{N-1} \sum_{m=0}^{N-1} a_n a_m t_n t_m (\mathbf{x}_n^T \mathbf{x}_m)$$

Subject to

$$a_n \geq 0 \text{ and } 0 = \sum_{n=0}^{N-1} a_n t_n$$

Note that the loss involves combinations or relations, between the datapoints $\mathbf{x}_n^T \mathbf{x}_m = \langle \mathbf{x}_n, \mathbf{x}_m \rangle$

This does not seem simpler, but it is as it allows us to use **kernels**!

Classifying points with an SVM (with dual representation)

To classify points with the trained model we evaluate

$$y(\mathbf{x}) = \sum_{n=0}^{N-1} a_n t_n \langle \mathbf{x}, \mathbf{x}_n \rangle + b$$

If $y(\mathbf{x}) \geq 0$ the datapoint is of the positive class. Otherwise it is neg. class.

Note 1: Most a_n are zero. The classifier depends only on a few **support vectors**.

Note 2: The inner product between the input data point and the n-th support vector $\langle \mathbf{x}, \mathbf{x}_n \rangle = \mathbf{x}^T \mathbf{x}_n$ can be replaced by a kernel to generate a non-linear classifier.

Kernels

Some questions

- On which ways you can measure similarity between two vectors?
- How could remove the influence of some “far away” values that you know that are less important?

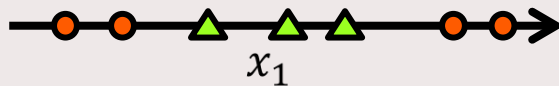
Non-linearly separable data

When data is not linearly separable, we can transform the data in such a way that we can use linear classifiers.

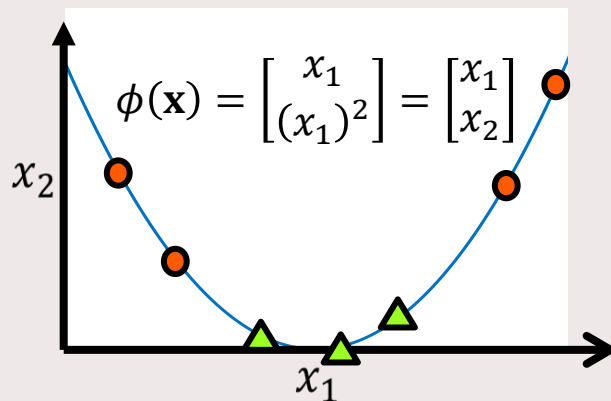
Think for example, to the design matrix in linear regression.

The mapping is defined by function $\phi(\cdot)$

Not linearly separable



Linearly separable



Problem with mappings

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \phi(\mathbf{x}) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ x_1 x_3 \\ x_2 x_1 \\ x_2 x_2 \\ x_2 x_3 \\ x_3 x_1 \\ x_3 x_2 \\ x_3 x_3 \end{bmatrix}$$

$$\mathbf{x} \in \mathbb{R}^n$$

$$\phi(\mathbf{x}) \in \mathbb{R}^{n^2}$$

What does it cost?

- Computing $\phi(\mathbf{x})$ is of order $O(n^2)$
- Since n is typically large (> 10.000), the amount of **required computation explodes** ($n^2 > 100.000.000$)

Trade-off: Good mapping $\phi(\mathbf{x})$

vs.

computational complexity

From mappings to kernels

A mapping function $\phi(\mathbf{x}, \mathbf{z})$ is a kernel $K(\mathbf{x}, \mathbf{z})$ if it complies with

$$\phi(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^T \phi(\mathbf{z})$$

If this is the case, we can replace inner products $\langle \mathbf{x}, \mathbf{z} \rangle = \phi(\mathbf{x})^T \phi(\mathbf{z}) = K(\mathbf{x}, \mathbf{z})$.

We don't need to compute the mapping over the entire dataset, but only the kernel operation 😊.

One exercise

- Given the mapping

$$\phi(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right) \left[\frac{1}{\sqrt{0!}} \left(\frac{x}{\sigma^2}\right)^0, \frac{1}{\sqrt{1!}} \left(\frac{x}{\sigma^2}\right)^1, \frac{1}{\sqrt{2!}} \left(\frac{x}{\sigma^2}\right)^2, \dots \right]$$

- Compute the product $\phi(x)^T \phi(z)$.

- Hint: Use the Taylor series:

$$\exp(x) = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

Kernels

Some popular kernels

Linear kernel	$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$
Radial Basis Functions (RBF)	$K(\mathbf{x}, \mathbf{z}) = \exp \left\{ -\frac{\ \mathbf{x} - \mathbf{z}\ ^2}{2\sigma^2} \right\}$
Polynomial	$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + 1)^p$
Sigmoid	$K(\mathbf{x}, \mathbf{z}) = \tanh(\kappa \mathbf{x}^T \mathbf{z} - \delta)$

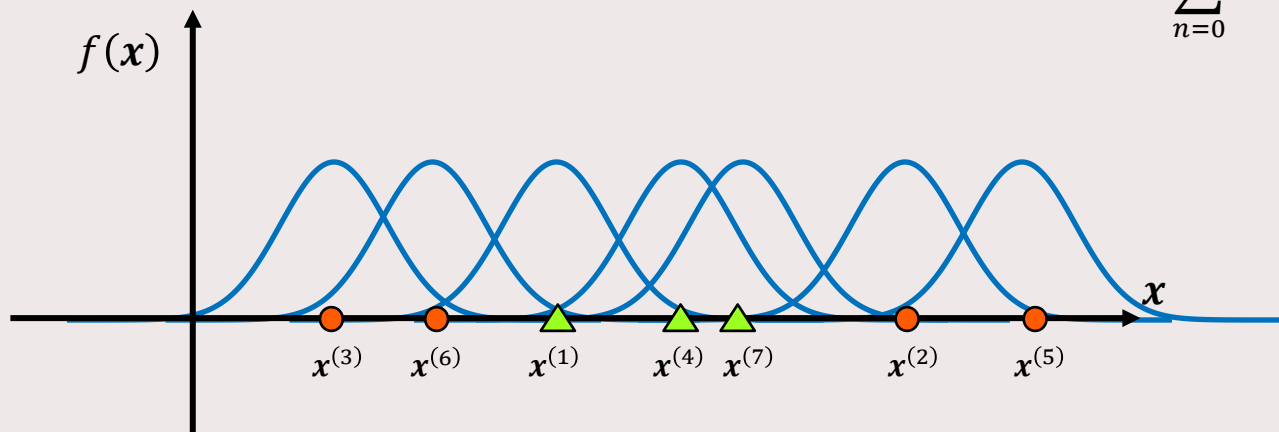
- You can apply kernels to all the methods we've seen so far!

Kernels

$$K(\mathbf{x}, \mathbf{z}) = \exp \left\{ -\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2} \right\}$$

How do kernels warp the decision function?

$$f(\mathbf{x}) = \sum_{n=0}^{N-1} \alpha_n t_n K(\mathbf{x}, \mathbf{x}_n) + b$$

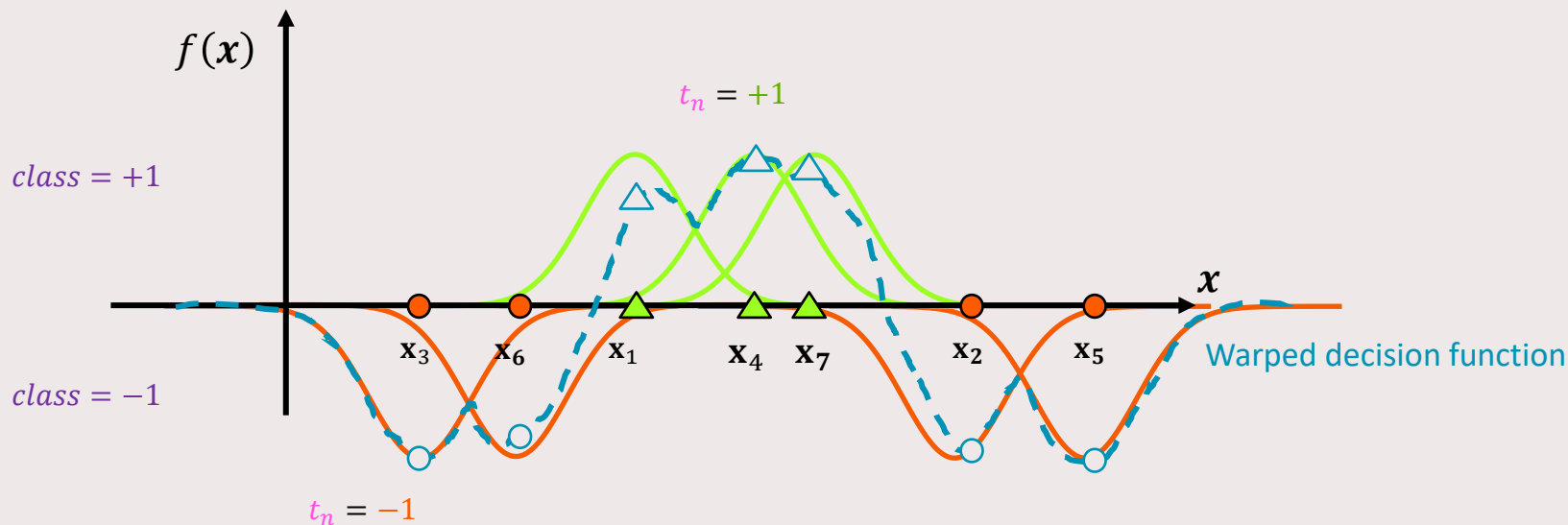


Kernels

$$K(\mathbf{x}, \mathbf{z}) = \exp \left\{ -\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2} \right\}$$

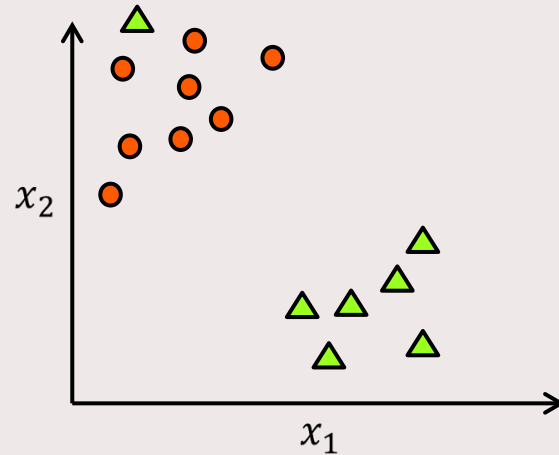
How do kernels warp the decision function?

$$f(\mathbf{x}) = \sum_{n=0}^{N-1} \alpha_n t_n K(\mathbf{x}, \mathbf{x}_n) + b$$



Soft MMC

Does this dataset allow to train successfully a hard maximum margin classifier?



Sample – pos. class



Sample – neg. class

What we have seen so far?

- **Hard maximum margin classifier**
 - There exists a classifier that can classify perfectly all the datapoints.
 - Primal formulation
 - Dual formulation (Uses representer theorem)
- **Kernel functions**
 - Map non-linear classification problem to linear one efficiently.

Overlapping data and MMC

Maximum margin classifier has the hard constraint $t_c(\mathbf{w}^T \mathbf{x}_c + b) \geq 1$

However, data often overlaps!

Soft MMC relaxes the distance constraint and allows to reduce (or even to cross) the margin

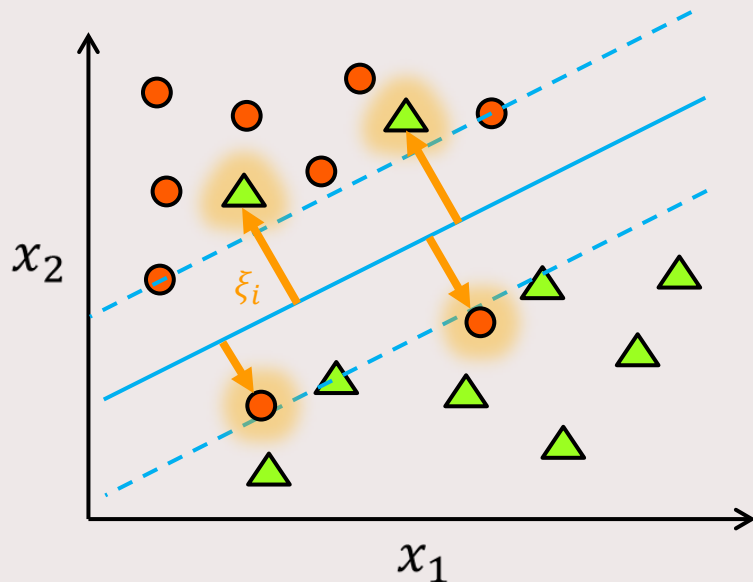
Support Vector Machine (SVM)

- We relax the margin constraint

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i$$

$$\text{s.t.} \quad t_n(\mathbf{w}^T \mathbf{x}_n + b) \geq 1 - \xi_n \quad n \in 0, \dots, N-1$$
$$\xi_n \geq 0$$

- ξ_i variables are called *slack* variables
- Functional margin can be < 1 , but it comes at a cost!
- **Soft-margin SVM**



Support Vector Machine (SVM)

Non-linear SVM: Add mapping function $\phi(x)$

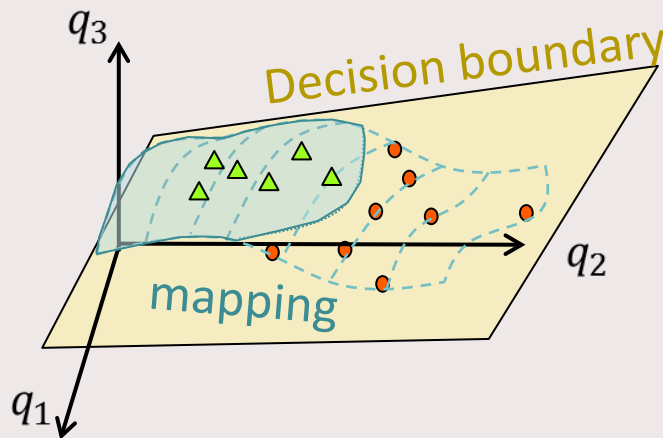
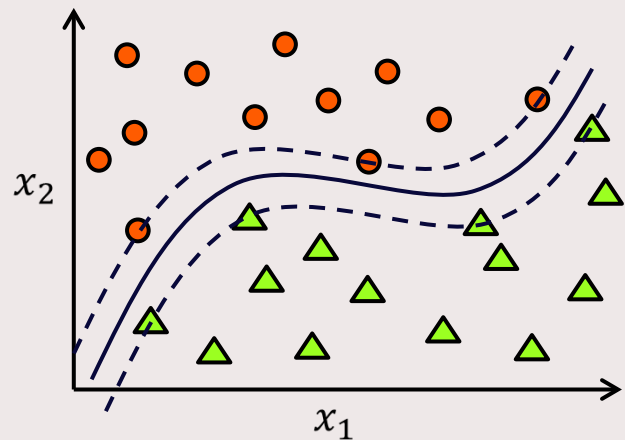
$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_n$$

s.t. $t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + b) \geq 1 - \xi_n$ $n \in 0, \dots, N - 1$

$$\xi_n \geq 0$$

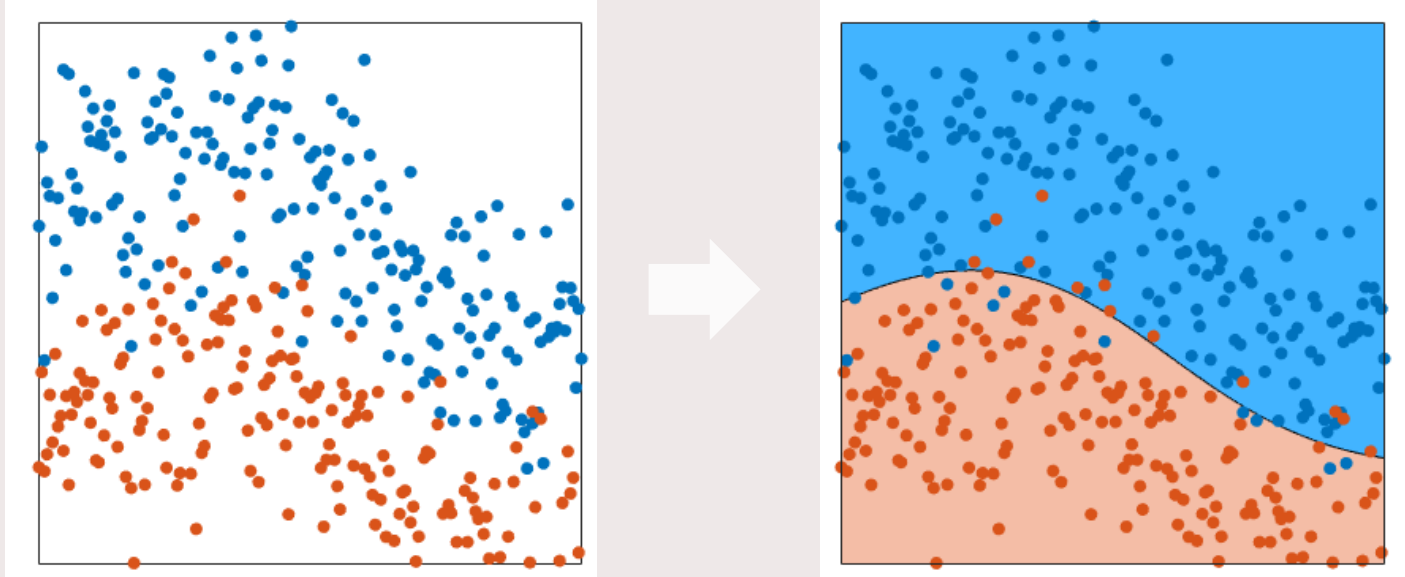
$\sum_{j=1}^N \alpha_n t_n K(\mathbf{x}, \mathbf{x}_n) + b \geq 1 - \xi_n$

Points $\mathbf{x}_n$ for which $\alpha_n \neq 0$ are the support vectors

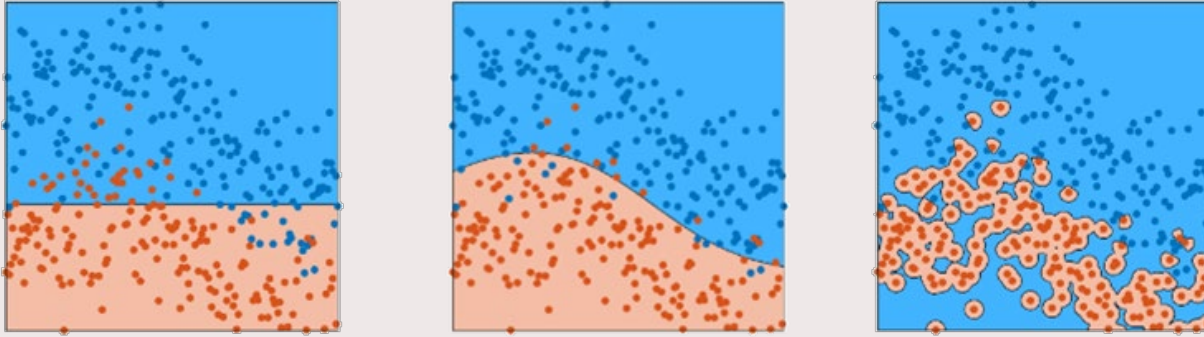


Support Vector Machine (SVM)

Results on non-separable, non-linear data



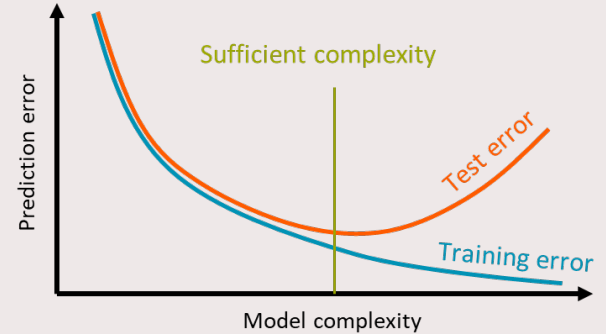
Support Vector Machine (SVM): Model complexity



Low complexity

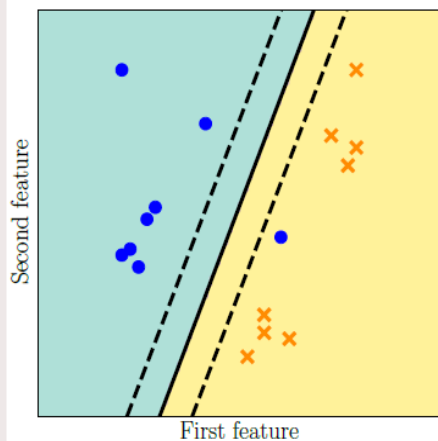


High complexity

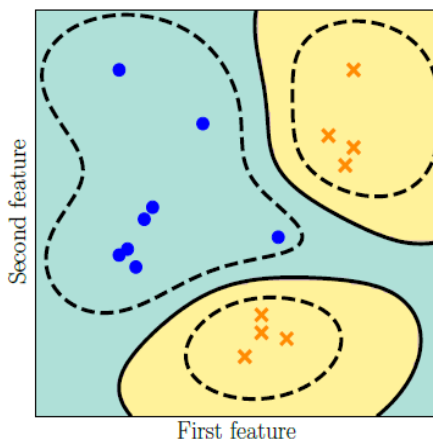


Support Vector Machine (SVM): Effect of different kernels

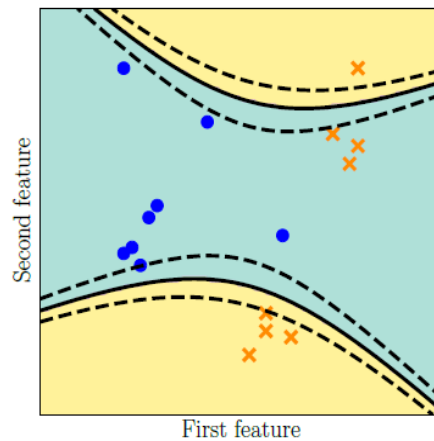
Figure 12.10 of "Mathematics for machine learning"



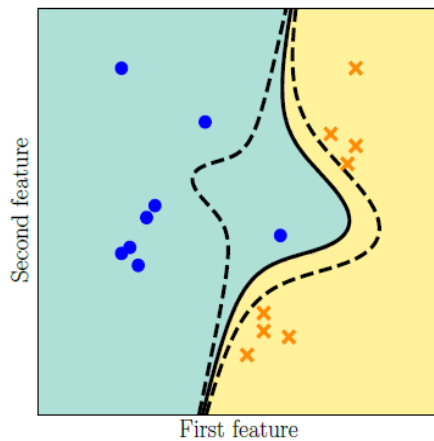
(a) SVM with linear kernel



(b) SVM with RBF kernel



(c) SVM with polynomial (degree 2) kernel



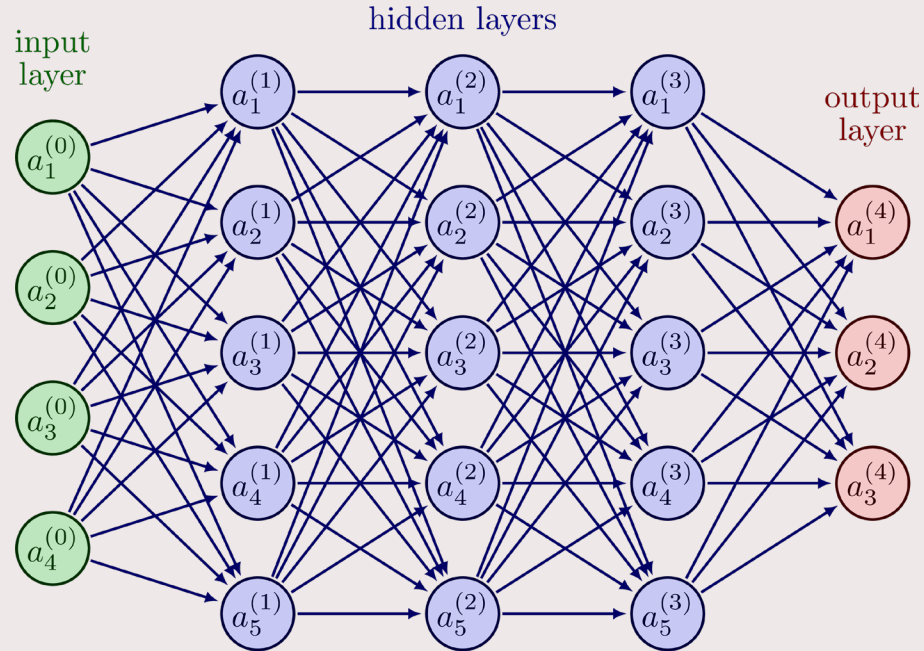
(d) SVM with polynomial (degree 3) kernel

Summary & look ahead

Summary

- **Maximum margin classifier (MMC)**
 - Increase the distance between decision boundary and data points.
- **Hard MMC vs Soft MMC**
 - Hard MMC requires that there is a line which can classify correctly all the points in the dataset.
 - Soft MMC relaxes this constraint by allowing some *slack*.
- **Kernels:** Allow to derive non-linear decision boundaries. Mappings also allow this, but kernels are more efficient at this.
- **Soft MMC + Kernel trick** is what we normally refer to as **Support Vector Machine (SVM)**

Next lecture



Additional materials on kernels

- **Video series of Statquest on support vector machines**
 - [Support Vector Machines Part 1 \(of 3\): Main Ideas!!!](#)
 - [Support Vector Machines Part 2: The Polynomial Kernel \(Part 2 of 3\)](#)
 - [Support Vector Machines Part 3: The Radial \(RBF\) Kernel \(Part 3 of 3\)](#)
- **Another good explanation on the mathematics of the derivation of the primal and dual problems for optimizing an SVM**
 - [Support Vector Machines: All you need to know! - YouTube](#)